Part 2 generate a new use case diagram:

Generated using google gemini and Plantuml.com

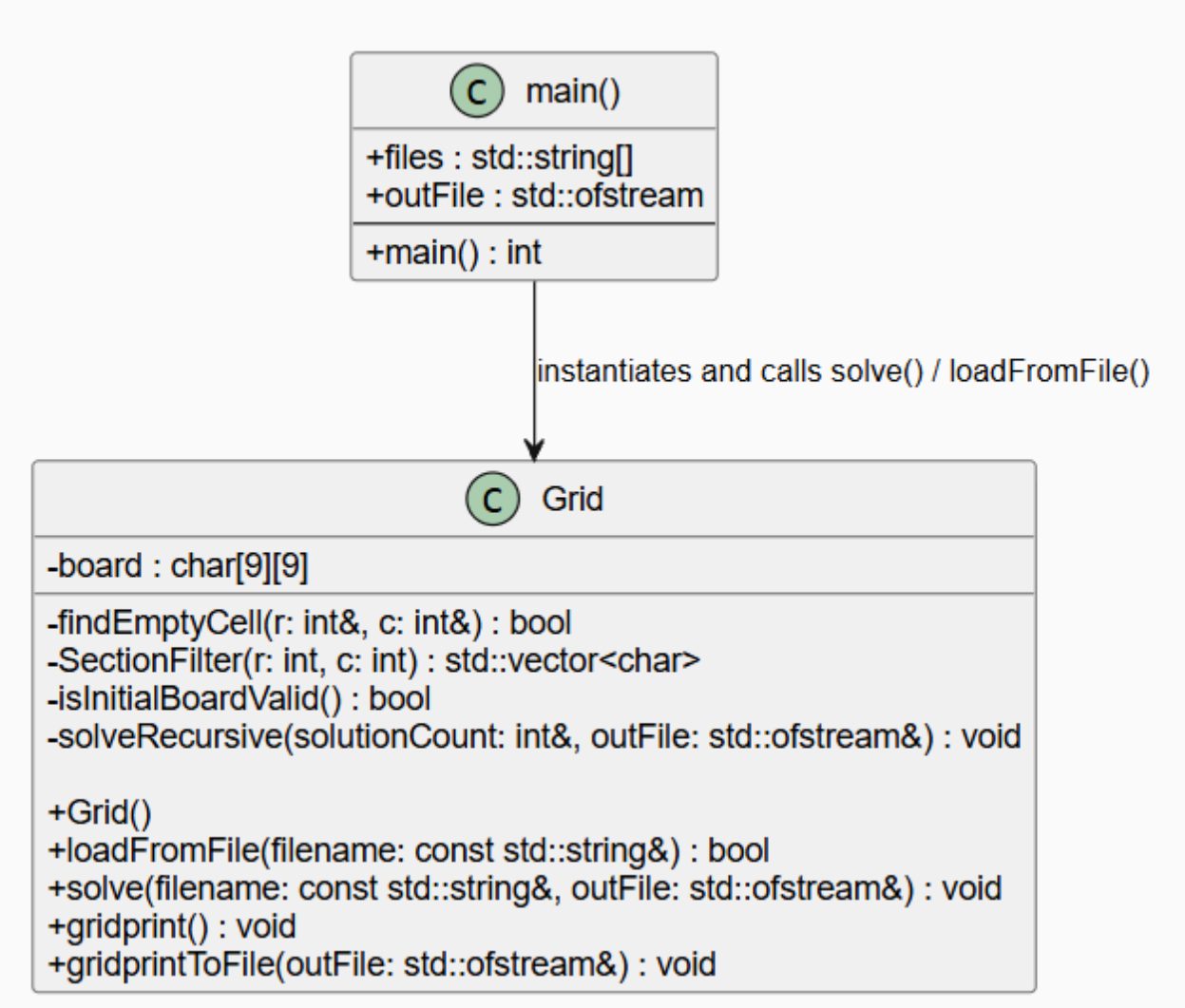


Diagram details:

Class Name: `Grid`

Attributes:

`- board : char[9][9]` (Private): A 2D character array representing the 9x9 Sudoku board state.

Private Methods:

`- findEmptyCell(r: int&, c: int&) : bool`: Locates the next available empty cell

('_') by reference.

- `SectionFilter(r: int, c: int) : std::vector<char>`: Computes and returns a list of valid candidate digits (`'1'` through `'9'`) for a given cell based on row, column, and 3x3 subgrid constraints.

- `isInitialBoardValid() : bool`: Validates that the pre-loaded puzzle contains no rule violations (duplicates) before solving.

- `solveRecursive(solutionCount: int&, outFile: std::ofstream&) : void`: The core backtracking algorithm that recursively fills cells and outputs solutions.

Public Methods:

+ `Grid()`: Default constructor that initializes all 81 cells to `'_'`.

+ `loadFromFile(filename: const std::string&) : bool`: Reads puzzle data from an input text file.

+ `solve(filename: const std::string&, outFile: std::ofstream&) : void`: Controller method that initiates validation and triggers the recursive solver.

+ `gridprint() : void`: Prints the current board state cleanly to the console.

+ `gridprintToFile(outFile: std::ofstream&) : void`: Writes the solved grid layout directly to the output stream.

-Interactions with other classes:

outside of interacting with the include objects this is the only class in the code, Its interactions with the Main() function are as follows:

Main instantiates the grid and calls the loadfromfile() function to interact with the grid. Their relationship is a direct association since they call each other.